Weak Session-Token Generation in an Android APK Due to the Use of `java.util.Random`

Rongbang Cheng *540506620* | Sienna Cai *540907399* | Yuzhi Shao *550139687* | Letian Chen *541012940* | Rongbang Cheng *540506620*

Abstract

We analyzed the provided Android APK and identified one vulnerability that fits the Assignment 1 randomness/cryptography scope: after successful credential verification, the app generates a value named `sessionToken` using `java.util.Random` and stores it in `SharedPreferences`. This is security-relevant because the token is created in the login flow, persisted as session state, later retrievable, and removed on logout. A session token therefore requires unpredictability, but `java.util.Random` is a deterministic non-cryptographic PRNG. Based on the APK evidence, the strongest defensible claim is not guaranteed remote takeover, but weakened unpredictability of authentication-related session state under realistic local-analysis conditions. The direct mitigation is to replace `java.util.Random` with `java.security.SecureRandom` in the token-generation path and preserve explicit session-token lifecycle controls.

1 Introduction

The provided APK contains three main activities: `MainActivity`, `Login`, and `Profile`. In the observed flow, `Login` verifies credentials and, on success, creates session-related state before opening `Profile`. We focus on one issue: weak generation of the authentication-related `sessionToken` in `Login.generateSessionToken()`. This fits the assignment scope because it is a misuse of randomness in a security-sensitive path rather than a cosmetic or UI-only use of randomness.

APK decompilation and static inspection are necessary to determine where security-sensitive values are generated, stored, and later reused. For randomness issues, the key question is not whether a value appears random, but whether the generator is suitable for its security role.

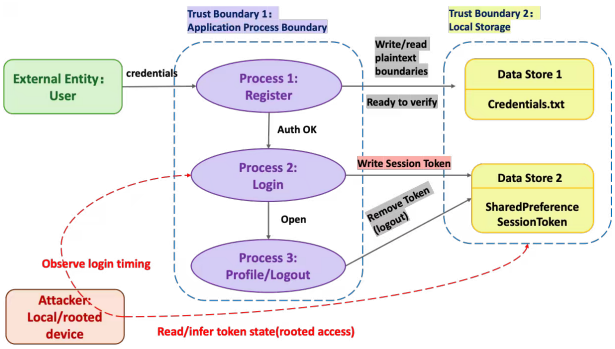

Figure 1: System architecture and attack surface for session-token generation and storage across trust boundaries.

2 System and Threat Model

The relevant components are `MainActivity`, which writes local credentials to `credentials.txt`; `Login`, which validates credentials and creates session state; and `Profile`, which represents the post-login state and clears session data on logout. The protected asset in the selected path is the unpredictability of the authentication-related `sessionToken`, together with the integrity of the session state that depends on it.

Figure 1 shows the relevant data flow across application and storage boundaries: after credential verification in `Login`, `createSession()` generates a `sessionToken`, stores it in `SharedPreferences` ("SessionPrefs"), and later retrieves or clears it through `getSessionToken()` and `clearSession()`.

This flow matters because the token is not a decorative random value. It is created immediately after successful authentication, explicitly labeled as `sessionToken`, persisted in session storage, and later treated as part of the authenticated-state lifecycle. That linkage is what makes the randomness quality security-relevant.

Our attacker model is intentionally bounded. We assume a local-analysis attacker operating in an emulator, rooted de-

vice, or instrumentation/debug environment. This attacker can inspect decompiled code, estimate the victim's login time within a bounded window, and may be able to inspect local application state or test token-dependent behavior. The attacker goal is not assumed remote compromise; rather, it is to reduce the uncertainty of a security-sensitive token enough to make offline reproduction or local comparison materially easier than intended. We do not assume backend compromise, remote code execution, or a proven downstream server-side token-validation sink, because the APK evidence does not justify those stronger claims.

3 How the Vulnerability Was Found

We decompiled the APK and searched for randomness usage associated with `random`, `token`, `session`, and `login`. This revealed two random-generation sites. The first, `MainActivity.randomNumberGenerator()`, appears UI-adjacent and has no clear security role. The second, `Login.generateSessionToken()`, is invoked by `createSession()` after successful login.

The second site is the stronger finding because `createSession()` stores the value under `sessionToken` in `SharedPreferences`, `getSessionToken()` retrieves it, and `Profile.clearSession()` removes it on logout. We therefore excluded the UI-only random value and focused on the authentication-related path.

4 Vulnerability Details and Exploitation Path

The generation logic is the core weakness. `generateSessionToken()` constructs a 16-character token from an alphanumeric alphabet, but the randomness source is `new Random()`, and the characters are chosen through repeated `nextInt(62)` calls. `java.util.Random` is a deterministic PRNG, not a cryptographically secure random generator. For an authentication-related session token, the required property is not merely a large-looking output space, but unpredictability against an attacker with partial knowledge about generation conditions.

A realistic exploitation path is therefore:

1. The attacker observes or closely estimates when the victim logs in under a realistic local-analysis setting.
2. The attacker re-implements the same `generateSessionToken()` logic offline and enumerates candidate outputs for plausible seed windows around that time.
3. If the attacker can inspect local `SharedPreferences` state, or can test behavior that depends on the token, the candidates can be compared against observed state or behavior.

This is a concrete path because it links attacker knowledge, generator determinism, and a comparison mechanism. At the same time, the evidentiary boundary must remain clear: the APK shows token creation, storage, retrieval, and clearing more clearly than any downstream token-validation enforcement. Therefore, the strongest defensible impact is weakened unpredictability of authentication-related session state, rather than a guaranteed authentication bypass, session hijack, or remote takeover. That bounded claim is still meaningful, because a value explicitly used as session state should not be credibly enumerable from a predictable PRNG.

5 Fix and Why It Works

The direct fix is to replace `java.util.Random` with `java.security.SecureRandom` in `Login.generateSessionToken()`. A corrected implementation should generate secure random bytes first and then encode them into a string representation suitable for storage and later use, rather than selecting characters through a deterministic general-purpose PRNG.

This fix works because it removes the exact property the attack relies on: credible output enumeration from a predictable generator under bounded timing assumptions. In the vulnerable design, an attacker can reason from approximate generation time to plausible seeds and then to candidate outputs. After replacing the generator with `SecureRandom`, that reasoning path is no longer technically sound in the same way.

Additional controls can further strengthen the design, including explicit token rotation on login, invalidation on logout, and explicit validation before granting access to protected flow. However, these should be presented as session-management hardening measures rather than as proof that the current APK already enforces token-based access correctly. Based on the present APK evidence, the strongest supported mitigation claim is that replacing `Random` with `SecureRandom` removes the predictability path identified in this report.

6 Conclusion

The APK generates an authentication-related `sessionToken` with `java.util.Random` immediately after successful login and stores it as session state. Because the token is part of the authentication lifecycle, its generation requires cryptographic unpredictability. The use of a deterministic non-CSPRNG in that role is therefore a genuine security design flaw. Our claim is intentionally bounded to what the APK evidence supports: weakened unpredictability of authentication-related session state under realistic local-analysis conditions. Replacing `Random` with `SecureRandom` is the direct and technically appropriate remediation.